

RELATÓRIO TÉCNICO E AVALIAÇÃO CRÍTICA

Disciplina: ECT-3401 — Computação Numérica

Unidade: I

Instituição: Escola de Ciências e Tecnologia — Universidade Federal do Rio Grande do Norte (UFRN)

Docente: Prof. Dr. Joilson B. A. Rego

Tema: Conversão de Bases e Operações Elementares: Um Estudo Prático em Ponto Flutuante Sem Coerção Decimal

Resumo

Este relatório documenta a concepção, fundamentação matemática, implementação e análise de um software desenvolvido para realizar operações aritméticas elementares (adição e subtração) e conversão direta entre sistemas de bases arbitrárias ($2 \leq \beta \leq 36$) para números reais ($x \in \mathbb{R}$). O diferencial central do projeto consiste na eliminação total de qualquer coerção para a base decimal (10) ou para tipos primitivos de ponto flutuante de máquina (padrão IEEE 754). Números são modelados através de uma estrutura vetorial customizada de dígitos inteiros isolados. A conversão da parte inteira é executada estritamente na base de destino via Algoritmo de Horner, enquanto a parte fracionária é convertida por multiplicações sucessivas calculadas estritamente na base de origem, ambas operacionalizadas exclusivamente através da aritmética de adição da respectiva base. Demonstra-se que o rastreamento discreto do histórico de estados fracionários elimina ruídos digitais de arredondamento e possibilita a identificação analítica de dízimas periódicas nativas.

1. Contexto e Motivação

A representação de grandezas numéricas na arquitetura dos computadores fundamenta-se predominantemente no padrão IEEE 754. Em ambientes de software convencionais, quando se deseja converter um valor expresso em uma base de origem arbitrária β_1 para uma base de destino β_2 (como converter da Base 7 para a Base 13), os compiladores e interpretadores tipicamente convertem o número para a representação decimal da máquina (int ou float) e, em seguida, realizam sucessivas divisões ou multiplicações para atingir a base β_2 .

Essa prática acarreta dois prejuízos técnicos:

1. **Mascaramento da Teoria Aritmética:** O estudante e o desenvolvedor perdem o contato com as propriedades fundamentais dos sistemas. O comportamento do transporte de dígitos (*carry* ou "vai-um"), do empréstimo posicional (*borrow* ou "emprresta-um") e da dinâmica polinomial desaparece sob as abstrações de baixo nível do processador.
2. **Erros de Arredondamento:** A passagem por um pivô binário/decimal de precisão finita gera dízimas periódicas decorrentes das incompatibilidades entre os fatores primos das bases. Por exemplo, o número 0.1_{10} (uma fração decimal finita com fator primo 5) transforma-se em uma dízima periódica infinita no formato IEEE 754 binário ($0.0001100110011 \dots_2$). O truncamento imposto pela mantissa finita do hardware introduz ruído que contamina a integridade de qualquer conversão subsequente para uma terceira base.

Diante desse cenário, este projeto adota a seguinte abordagem: os tipos primitivos são substituídos por estruturas de dados que armazenam vetores de dígitos discretos, manipulados exclusivamente por algoritmos fundamentados na teoria dos números e na álgebra posicional.

2. Objetivos

2.1 Objetivo Geral

Construir em linguagem Python 3.11+ um software capaz de executar operações aritméticas elementares (adição e subtração) e realizar conversão direta entre quaisquer bases arbitrárias no domínio dos reais ($x \in \mathbb{R}$) sem qualquer intermediação decimal ou coerção IEEE 754.

2.2 Metas Específicas

- **O1:** Desenvolver algoritmos de conversão direta entre bases arbitrárias manipulados estritamente na base de origem ou de destino via aritmética polinomial.
 - **O2:** Implementar mapeamento dinâmico de caracteres para suportar dígitos e letras (0–9, A–Z).
 - **O3:** Garantir a ausência total da base 10 como pivô durante as transformações numéricas.
 - **O4:** Efetuar operações elementares diretamente na base selecionada, detalhando contas com alinhamento e rastreo de *carry* e *borrow*.
 - **O5:** Identificar dízimas periódicas na base de destino a partir do histórico de restos gerados na base de origem.
-

3. Fundamentação Teórica e Algorítmica

3.1 Representação Posicional Polinomial

Qualquer número real X pode ser expresso em uma base inteira $\beta \geq 2$ pela série posicional:

$$X = \pm \left(\sum_{i=0}^n d_i \cdot \beta^i + \sum_{j=1}^{\infty} d_{-j} \cdot \beta^{-j} \right)$$

onde cada dígito satisfaz $d_k \in \{0, 1, \dots, \beta - 1\}$.

3.2 Conversão da Parte Inteira

Para converter a parte inteira $(d_n d_{n-1} \dots d_0)_{\beta_1}$ para a base β_2 , avalia-se o polinômio na base β_2 utilizando o método aninhado de Horner:

$$P_0 = (d_n)_{\beta_2}$$

$$P_k = (P_{k-1} \times \beta_1) + (d_{n-k})_{\beta_2}, \quad k = 1, \dots, n$$

Rigor de não-coerção:

- A base β_1 é tratada como uma quantidade representada na base β_2 .
- O produto $P_{k-1} \times \beta_1$ é computado utilizando o algoritmo *double-and-add* baseado na função de **adição na base β_2** .
- O dígito (d_{n-k}) é somado utilizando a mesma rotina de adição na base β_2 .

3.3 Conversão da Parte Fracionária

Para a parte fracionária $F_0 = (0.d_{-1}d_{-2} \dots d_{-m})_{\beta_1}$, a conversão para β_2 opera no sentido inverso, manipulada na base de origem β_1 : A cada iteração $k \geq 1$:

$$W_k = F_{k-1} \times \beta_2 \quad (\text{calculado na base } \beta_1)$$

O valor que transborda a vírgula para a ordem inteira, $I_k = \lfloor W_k \rfloor$, representa o k -ésimo dígito na base de destino β_2 . O restante fracionário torna-se:

$$F_k = W_k - I_k = (W_k) \pmod{1}$$

Novamente, o produto $F_{k-1} \times \beta_2$ é obtido somando F_{k-1} sucessivamente β_2 vezes na própria base β_1 .

3.4 Teorema da Periodicidade e Detecção de Dízimas

Uma fração irredutível p/q possui representação finita na base β se, e somente se, todos os fatores primos de q dividem β . Caso contrário, a representação torna-se inevitavelmente uma **dízima periódica**.

Ao rastrear os estados do vetor de dígitos fracionários F_k na base de origem, se em uma iteração k encontrarmos um estado F_k idêntico a um estado prévio F_j ($j < k$), o processo garante a

existência de um ciclo periódico de comprimento $T = k - j$. O padrão repetitivo é composto pelos dígitos gerados entre $j + 1$ e k .

4. Modelagem e Arquitetura

O sistema foi estruturado de forma modular e desacoplada:

1. **Alphabet (src/alphabet.py):** Encapsula a correspondência entre o caractere textual (ex: '0'-'9', 'A'-'Z') e o índice inteiro associado, suportando bases de 2 a 36 com validação.
 2. **BaseNumber (src/base_number.py):** Objeto do projeto. Armazena o sinal (± 1), um vetor para dígitos inteiros `integer_digits: List[int]`, um vetor para dígitos fracionários `fractional_digits: List[int]` e um ponteiro opcional para o início do período `repeating_start`. A classe implementa normalização automática (eliminação de zeros supérfluos) e comparação de magnitude (`abs_compare`).
 3. **Operations (src/operations.py):**
 - **add(a, b):** Realiza o alinhamento preenchendo zeros à direita (frações) e à esquerda (inteiros). Executa a soma coluna a coluna da direita para a esquerda. Em caso de sinais contrários, desvia para a subtração de magnitudes. Produz uma estrutura `AdditionTrace` que documenta os *carries* de cada coluna.
 - **subtract(a, b):** Realiza o alinhamento e calcula a diferença de magnitudes. Se o minuendo é menor que o subtraendo, permuta a ordem e inverte o sinal do resultado. O algoritmo rastreia os empréstimos em cadeia e gera uma estrutura `SubtractionTrace` com os *borrows* de cada coluna.
 - **scalar_multiply(num, scalar):** Multiplica um `BaseNumber` por uma constante inteira através do método de duplicação e soma (*double-and-add*), invocando iterativamente a própria função `add`.
 4. **Converter (src/converter.py):** Implementação do método de Horner e do método dos restos descritos na Seção 3, registrando em `ConversionResult` cada transição de estado e o diagnóstico de periodicidade.
 5. **Display (src/display.py):** Formata a saída no terminal, apresentando o cabeçalho posicional, linha de vai-um / empresta-um, operandos alinhados e linha divisória.
-

5. Resultados Experimentais e Avaliação

A seguir são apresentados experimentos que evidenciam o funcionamento do software e comprovam as hipóteses levantadas no projeto.

Experimento 1: Operações Elementares e Contas Armadas com Propagação de Carries e Borrows

Caso 1.1: Adição Hexadecimal com Múltiplos Carries em Cascata

Operação: $(1F.8)_{16} + (0.C)_{16}$

--- CONTA (ADIÇÃO NA BASE 16) ---

```
[vai-um]      11
              1F.8
            +   00.C
            -----
              20.4
```

Passo a passo por coluna (da direita para a esquerda):

- Fracionário col 1: $8 (8) + C (12) = 20 \rightarrow$ Dígito: 4 (4), Novo vai-um: 1
- Inteiro col 0: $F (15) + 0 (0) + \text{vai-um} (1) = 16 \rightarrow$ Dígito: 0 (0), Novo vai-um: 1
- Inteiro col 1: $1 (1) + 0 (0) + \text{vai-um} (1) = 2 \rightarrow$ Dígito: 2 (2), Novo vai-um: 0

Análise: O fracionário $8_{16} + C_{16} = 8 + 12 = 20_{10} = 1 \times 16 + 4$ gerou o dígito 4 e um *carry* de valor 1 através da vírgula. Na coluna inteira $0, F_{16} + 1 = 16_{10} = 10_{16}$, gerando dígito 0 e propagando novo *carry* para a coluna 1, resultando no valor exato $(20.4)_{16}$.

Caso 1.2: Subtração Octal com Empréstimo em Cascata Através de Zeros

Operação: $(100.2)_8 - (0.7)_8$

--- CONTA (SUBTRAÇÃO NA BASE 8) ---

```
[empresta]    111
              100.2
            -   000.7
            -----
              77.3
```

Passo a passo por coluna (da direita para a esquerda):

- Fracionário col 1: $2 (2) + \text{base} (8) [\text{pediu emprestado}] - 7 (7) = 3 (3)$
- Inteiro col 0: $0 (0) - \text{empresta} (1) + \text{base} (8) [\text{pediu emprestado}] - 0 (0) = 7 (7)$
- Inteiro col 1: $0 (0) - \text{empresta} (1) + \text{base} (8) [\text{pediu emprestado}] - 0 (0) = 7 (7)$
- Inteiro col 2: $1 (1) - \text{empresta} (1) - 0 (0) = 0 (0)$

Análise: Evidencia o comportamento do *borrow* atravessando dígitos zero. Como a base é 8, o empréstimo adiciona o peso 8 à posição corrente e decrementa 1 da posição imediatamente superior, reproduzindo com exatidão o raciocínio manual de registradores.

Experimento 2: Conversão Direta entre Bases Não-Decimais (Base 7 para Base 13)

Conforme proposto nominalmente na introdução do projeto pelo docente: converter valores da Base 7 diretamente para a Base 13.

Operação: Converter $(25)_7$ para a Base 13.

- Na base 7: $2 \times 7^1 + 5 \times 7^0 = 19_{10}$.
- Na base 13: $19 = 1 \times 13^1 + 6 \times 13^0 = (16)_{13}$.

Execução no software:

```
=====
RELATÓRIO DE CONVERSÃO DIRETA DE BASE (SEM COERÇÃO DECIMAL)
=====
Número de Origem : 25 (Base 7)
Base de Destino  : Base 13
Resultado Final  : 16 (Base 13)
-----
[FASE 1] Conversão da Parte Inteira (Horner na Base de Destino):
  • Inicialização do acumulador na base de destino (13): P = 0
  • Passo 1 (dígito de peso 7^1 = '2' [valor 2]): P = (P * 7) + 2 => P = 2
  • Passo 2 (dígito de peso 7^0 = '5' [valor 5]): P = (P * 7) + 5 => P = 16
-----
[FASE 2] Conversão da Parte Fracionária (Multiplicações na Base de Origem):
  • Parte fracionária nula; conversão direta imediata.
-----
[FASE 3] Diagnóstico de Periodicidade Fracionária:
  Status      : Número estritamente inteiro.
=====
```

Análise: O acumulador inicializou em 0 na base 13. No passo 1, $P = 0 \times 7 + 2 = 2$. No passo 2, $P = 2 \times 7 + 5$. Na base 13, $2 \times 7 = 14_{10} = (11)_{13}$. Somando 5_{13} a $(11)_{13}$ via adição na base 13, obtém-se $(16)_{13}$, sem nenhuma passagem por variáveis float ou primitivas do sistema.

Experimento 3: Detecção Exata de Dízimas Periódicas Nativas

Caso 3.1: Conversão Fracionária de Base 4 para Base 3

Operação: Converter $(0.12)_4$ para a Base 3.

- Analiticamente: $(0.12)_4 = \frac{1}{4} + \frac{2}{16} = \frac{6}{16} = \frac{3}{8} = 0.375_{10}$.
- Em base 3: $\frac{3}{8} = 0.\overline{10}_3$ (período de tamanho 2 com dígitos 1 e 0).

Execução no software:

```
=====
RELATÓRIO DE CONVERSÃO DIRETA DE BASE (SEM COERÇÃO DECIMAL)
=====
```

Número de Origem : 0.12 (Base 4)
Base de Destino : Base 3
Resultado Final : 0.(10) (Base 3)

[FASE 1] Conversão da Parte Inteira (Horner na Base de Destino):

- Inicialização do acumulador na base de destino (3): $P = 0$
- Passo 1 (dígito de peso $4^0 = '0'$ [valor 0]): $P = (P * 4) + 0 \Rightarrow P = 0$

[FASE 2] Conversão da Parte Fracionária (Multiplicações na Base de Origem):

- Início da conversão fracionária: Fração inicial = 0.12 na base 4.

Multiplicador = base de destino (3).

- Passo 1: Fração atual * 3 = 1.02 (base 4) -> Dígito extraído: '1' (valor 1),
Nova fração restante: 0.02

- Passo 2: Fração atual * 3 = 0.12 (base 4) -> Dígito extraído: '0' (valor 0),
Nova fração restante: 0.12

- Passo 3: Estado fracionário (1, 2) já ocorreu no passo 1. Dízima periódica nativa detectada! Período de tamanho 2: (10).

[FASE 3] Diagnóstico de Periodicidade Fracionária:

Status : DÍZIMA PERIÓDICA NATIVA IDENTIFICADA!

Início do ciclo : dígito 1 após a vírgula

Comprimento (T) : 2 dígitos

Padrão repetido : (10)

=====

Análise Crítica: O algoritmo identificou o retorno ao estado fracionário inicial (1, 2) imediatamente no Passo 3. Se essa conversão tivesse sido realizada convertendo para o tipo primitivo `float(0.375)` em binário IEEE 754, a precisão dependeria da convergência da mantissa e a identificação do ciclo exigiria comparações por tolerância ϵ , sujeitas a falso-positivos ou detecção de períodos espúrios. Aqui, a igualdade do vetor (1, 2) é analiticamente exata.

Caso 3.2: O Paradigma do IEEE 754 (0.1_{10} em Binário)

Operação: Converter 0.1_{10} para a Base 2. Execução no software:

Resultado Final: 0.0(0011) (Base 2)

Fase 3:

Status : DÍZIMA PERIÓDICA NATIVA IDENTIFICADA!

Início do ciclo : dígito 2 após a vírgula

Comprimento (T) : 4 dígitos

Padrão repetido : (0011)

Análise Crítica: O software confirma formalmente o motivo pelo qual computadores binários sofrem com o número 0.1: trata-se de uma dízima periódica composta por um dígito não-periódico (0) seguido de um ciclo infinito de 4 bits (0011). O algoritmo deduziu a dízima com apenas 5 multiplicações exatas na base 10, demonstrando a superioridade da álgebra posicional pura para fins didáticos e analíticos.

Experimento 4: Tabela Comparativa de Abordagens

Métrica / Critério	Abordagem Convencional (Pivô IEEE 754)	Abordagem Desenvolvida (Estrutura Vetorial Pura)
Pivô intermediário	Obrigatório (Base 2 / Base 10)	Inexistente (conversão direta)
Deteção de dízimas	Heurística / Aproximada com ϵ	Analítica e Exata (comparação de tuplas)
Limitação de precisão	53 bits de mantissa (~15 a 17 dígitos)	Ilimitada (vetores dinâmicos em memória)
Visibilidade de carries/borrows	Ocultada pelo silício da CPU	Visível e Auditável (passo a passo coluna por coluna)
Fidelidade da conta armada	Requer reconstrução artificial	Nativa e Fidedigna ao cálculo posicional

6. Conclusões

O projeto atingiu com êxito todos os objetivos propostos para a Unidade I da disciplina de Computação Numérica:

- 1. Eficiência Algorítmica e Pureza Teórica:** A combinação do Algoritmo de Horner calculado na base de destino com o método das multiplicações sucessivas na base de origem provou que qualquer conversão entre bases pode ser realizada sem nunca invocar a base decimal ou coerções binárias de hardware.
- 2. Didática e Transparência Aritmética:** As operações de adição e subtração desenvolvidas registram fielmente os mecanismos de *carry* e *borrow*, viabilizando a renderização de contas armadas idênticas às efetuadas manualmente sobre o papel, suprimindo a carência de visibilidade da teoria aritmética.
- 3. Robustez na Análise Numérica:** A estratégia de armazenar tuplas de dígitos fracionários na base de origem possibilitou isolar dízimas periódicas nativas sem contaminação por erros de truncamento de ponto flutuante.

Em suma, a metodologia adotada consolida os princípios dos sistemas de numeração e fornece uma ferramenta pedagógica e computacional robusta para o estudo de aritmética de precisão arbitrária.

7. Referências Bibliográficas

1. **KNUTH, Donald E.** *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. 3rd ed. Boston: Addison-Wesley, 1997.
2. **GOLDBERG, David.** *What Every Computer Scientist Should Know About Floating-Point Arithmetic*. ACM Computing Surveys (CSUR), v. 23, n. 1, p. 5–48, 1991.
3. **RUGGIERO, Márcia A. G.; LOPES, Vera Lúcia da Rocha.** *Cálculo Numérico: Aspectos Teóricos e Computacionais*. 2. ed. São Paulo: Pearson Makron Books, 1996.
4. **BURDEN, Richard L.; FAIRES, J. Douglas.** *Análise Numérica*. 8. ed. São Paulo: Cengage Learning, 2008.
5. **IEEE.** *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019, p. 1–84, 2019.